胡傲东

18085741680 hu.aodong@icloud.com GitHub 个人主页

核心竞争力

复合型研发工程师，具备从跨端交互、Go/Python 服务到多模态 AI 应用落地的完整工程视角，能够在资源受限与复杂协同场景下完成架构拆分、性能治理与交付闭环。长期关注任务调度、系统性能与端云协同，能够把原型系统推进为可演示、可部署、可持续迭代的产品形态。

C++ / Qt · Go · TypeScript / React · Python / AI · 任务调度 · 系统性能

教育背景

华中科技大学 · 集成电路设计与集成系统（本科） 2022.09 - 2026.06

专业技能

语言与框架
C++17 / Qt、Go / Gin、Python / FastAPI、TypeScript / React 18 / Tauri

架构与交付
分层架构、RESTful API、异步任务调度、任务状态机、CI/CD、跨端协作

AI 应用工程
Retrieval-Augmented Generation (RAG)、GraphRAG、Tool Calling、多模态工作流、提示词编排、推理服务封装

系统与平台
Linux、TCP/IP、Redis、Asynq、MySQL、MinIO、FFmpeg

性能治理
并发资源治理、缓存复用、内存优化、长任务恢复、可观测性、故障补偿

项目经历

端到端 AI 视频生成系统（跨平台多模态任务平台） ↗ 全栈

字节剪映 · 工程营实习生小组长 · 2025.11 - 至今

项目定位：将文本生成、分镜规划、图像/配音/视频推理到 FFmpeg 合成的串行工作流重构为桌面端 + 服务端协同系统，带领 5 人在 2 周内交付 MVP，并在 8GB 显存下稳定生成 30s 720p 视频
架构取舍：多模态任务混跑时，常规 FIFO 队列与同步编排会放大 GPU/内存争用并拖垮尾任务；因此拆分 API 层、调度层、推理层，采用 Redis + Asynq 优先级队列与令牌桶限速，按任务类型、显存占用和时延敏感度动态调度
机制设计：定义创建/执行/暂停/恢复/取消/重试/降级的统一状态机，引入 checkpoint-resume 与素材指纹缓存，避免单阶段失败触发全链路重跑，长任务恢复率 95%+，缓存复用率从 25% 提升至 65%
端侧优化：针对实时预览阶段的 UI 卡顿与线程阻塞，采用 Qt/QML MVVM 分层、QThread + Signal-Slot 线程解耦与帧缓冲复用，内存峰值降低 30%，渲染帧率从 30fps 提升至 60fps
技术栈：Qt/QML/C++ · Go/Gin · FastAPI · Redis/Asynq · Docker · FFmpeg · Qwen / Stable Diffusion / Stable Video Diffusion / Text-to-Speech · MySQL/MinIO

AI 智能教学平台 ↗ 全栈

本科毕业设计 · 独立全栈开发 · 2025.11 - 至今

项目定位：将以课程为中心的传统教学平台重构为学生中心的 AI 教学系统，支撑桌面端、企业微信 H5 与移动端统一协同，在华中科技大学课程试点覆盖 200+ 研究生，稳定支持 30+ 并发用户
架构取舍：原方案将学情状态散落在课程维度表结构中，无法支撑跨课程追踪与个性化干预；因此重建学生画像、学习档案与长期学情模型，并将 React 前端、Go 服务与 Python AI 服务拆为可独立演进的三层系统
AI 机制：为降低专业问答 hallucination 与检索错配，放弃仅靠关键词/向量召回的轻量方案，引入 Qwen3 + GraphRAG + Tool Calling，把知识点检索、学情查询与建议生成拆成可追踪链路，AI 答疑准确率提升至 85%+
工程落地：围绕作业批改与高频答疑设计 Redis 缓存、异步数据同步与 Docker Compose 一键部署，兼顾教师试点环境的低运维成本与跨端交付效率，作业批改覆盖 80% 常见题型
技术栈：Vite/React/TypeScript/Tauri · Go/Gin/GORM · Python/FastAPI · Redis · MySQL/MinIO · Docker Compose · Qwen3 · GraphRAG · NPU acceleration

macOS 系统优化工具（SwiftSweep） ↗ Apple

个人独立项目 / 开源工具 · 2024.10 - 2024.12

项目定位：为解决 macOS 用户存储空间焦虑与常见清理工具高授权风险，基于 Swift 5.9 与 SwiftUI 独立开发集系统深度清理、相似文件检测与特权隔离于一体的原生维护工具（GUI + CLI），实现了 14+ 核心系统维护功能并成功完成 Apple 官方公证发布
架构与安全设计：摒弃传统清理软件索要全局 root 权限的危险模式，采用 XPC Helper + SMAppService 构建进程间通信隔离层，主应用在沙盒与受限权限下运行，彻底阻断了提权漏洞攻击面；设计了 dry-run 预演、基线白名单与严格的 symlink（符号链接）逃逸检测三重防护体系，实现 100% 零误删系统核心进程数据
算法与性能调优：面对数以万计的相似图片与视频检测导致的内存暴涨与 CPU 满载问题，引入基于 pHash（感知哈希）与 LSH（局部敏感哈希）降维检测算法；利用 Swift Task / Async 机制实现流式处理与内存回收，将重复媒体文件扫描速度提高了 3 倍，内存峰值控制在 200MB 以内
技术栈：Swift 5.9 / SwiftUI · XPC 进程间协议通信 · SMAppService · pHash / LSH 特征匹配算法 · Swift 包管理器 (SPM) · Developer ID 证书体系与 Notarization

基於 RISC-V 架构的五级流水线 CPU 设计 ↗ 系统

第七届全国大学生集成电路创新创业大赛（杯赛） · 核心开发 · 2023.01 - 2023.06

项目定位：从零开始使用 Verilog 基于 RISC-V (RV32I) 指令集架构，设计并实现了一颗完整的五级流水线 CPU，并成功在 FPGA 开发板上完成综合与功能验证，荣获全国总决赛二等奖
架构与冒险解决：独立完成了取指(IF)、译码(ID)、执行(EX)、访存(MEM)、回写(WB)核心数据通路的设计；面对流水线架构固有的数据冒险与控制冒险，设计了 Forwarding Unit（数据旁路转发单元）与 Hazard Detection Unit（流水线停顿检测单元），将核心运算指令在无冒险情况下的 CPI (每指令周期数) 稳定在 1.0
功能验证与完备性：构建了包含 50+ 汇编指令集的定制化测试激励（Testbench），涵盖了边界条件与复杂的分支跳转回归场景。通过 ModelSim 进行了指令级时序仿真验证，确认所有算术运算与跳跃指令行为符合 RISC-V 官方规范，保证了流片前 RTL 代码的高门级综合成功率
技术栈：Verilog HDL · RISC-V (RV32I) ISA · ModelSim / Vivado (仿真与综合) · FPGA 硬件开发

华为海思网络仿真平台重构与效能提升 系统

启明学院 Dian 团队 · 华为海思组核心研发 · 2023.03 - 2024.04

项目定位：主导华为海思内部底层网络仿真核心算法从自研平台向标准开源 NS-3 仿真环境的跨平台重构与迁移，保障了芯片研发与验证流程的不中断跨端迁移，将内部验证吞吐效率提升并深度融入了业界标准组件流
架构与特性适配：完成平台底层自底向上的跨版本重构（从 ns-3.31 强制演进至 ns-3.38），深入解析 Event Scheduler (事件调度器) 与 Time Model (时间步进模型) 内核代码。攻克了 C++ ABI 兼容性与内核对象生命周期管理痛点，大规模引入现代 C++ 智能指针体系接管手写内存释放，彻底消除了历史遗留的内存

泄漏风险

工程化指标建设：针对复杂网络拥塞与拓扑场景，独立设计并开发了高精度的网络性能统计观测节点（Metrics Sink Modules），实现了对端到端时延、实时吞吐率、丢包率及多级队列占用率的毫秒级无切入式探针采集；在十万级并发包规模下，其统计开销占比低于总仿真时长的 5%，极大加速了拥塞控制算法与 PHY/MAC 层协议的调优迭代周期

技术栈：C++ (Modern C++ / STL) · ns-3 Discrete-Event Simulator内核机制 · Python 脚本自动化编排 · Linux (GDB/Valgrind 性能剖析) · TCP/IP 协议栈及多层拥塞模型